

Badger Code / OpenAgent-Coding Proje Raporu

Bu rapor, projeye başlama ve öğrenme amaçlı karar verme dokümanıdır. Yarışma sayfasındaki deadline, geçerli kurallar, model kataloğu ve fiyatlar zamanla değişebileceğinden, uygulamaya başlamadan hemen önce Kaggle yarışma sayfası, [RESOURCES.md](#) ve Harbor dokümantasyonu yeniden kontrol edilmelidir.

1. Yönetici Özeti

Badger Code, açık-ağırlıklı bir büyük dil modeliyle gerçek yazılım mühendisliği işlerini çözebilen otonom bir coding agent geliştirme hackathon'ıdır. Amaç yalnızca bir model çağırmak değil; modelin terminali güvenli ve sistematik biçimde kullanmasını, hata çıktılarından öğrenmesini, planını gerektiğinde düzeltmesini ve izole Docker görevlerinde sonucu doğrulamasını sağlayan iyi bir agent mimarisi kurmaktır.

Bu proje, yarışmayı kazanma baskısı olmadan LangChain, Harbor/Terminal-Bench ve açık-ağırlıklı model serving'i tek, ölçülebilir bir uygulamada öğrenmek için çok uygun görünmektedir. Mevcut Mac, Docker/Harbor tarafında GPU'suz çalışabilir; model üretimi Nebius Token Factory endpoint'inde yapılır. Bu nedenle proje için ayrı GPU kiralamak yerine mevcut Nebius kredisi, LangChain kredisi ve Tavily erişimiyle odaklı bir prototip ve ardından ciddi bir benchmark çalışması yapmak mantıklıdır.

2. Projenin Amacı ve İçeriği

Kaggle'daki **Badger Code / OpenAgent-Coding** community hackathon'ı, ML+X, University of Wisconsin-Madison tarafından düzenlenmektedir. Yarışmanın temel problemi şudur: Açık-ağırlıklı bir LLM kullanarak, Terminal-Bench 2.1 içindeki 89 gerçekçi yazılım mühendisliği görevini çözebilen bir ajan üretmek.

Terminal-Bench 2.1, klasik çoktan seçmeli kod sorularından farklıdır. Her görev, ayrı bir Docker container içinde verilen gerçek bir çalışma ortamıdır. Örnekler arasında bozulmuş bir Git geçmişini kurtarma, bir derleme hatasını teşhis edip düzeltme, servis veya paket kurulumu, testleri geçirme, dosya ve sistem ayarlarını düzeltme gibi işler bulunur. Ajanın yalnızca doğru cevabı yazması yetmez: terminal komutları üretmesi, çıktıyı okuması, gerektiğinde başka komutlar çalıştırması ve ortamı görev testlerinin kabul edeceği son hâle getirmesi gerekir.

Bu görevleri çalıştırma ve değerlendirme katmanı **Harbor**'dır ([harbor-framework/harbor](#)). Harbor, görevleri tekrar üretilebilir şekilde hazırlayan, her deneme için temiz container başlatan, ajan ile ortam arasındaki etkileşimi yöneten ve başarıyı ölçen resmi framework olarak düşünülebilir. Böyle bir çerçevede, "ajan ekranda iyi görünüyor" izlenimi yerine aynı koşullarda tekrar çalıştırılabilen bir başarı metriği sağlar.

Yarışmanın ilginç tarafı, model boyutundan çok agent scaffold'unu sınamasıdır. Güçlü bir model, kötü tasarlanmış araç kullanımı, bağlam yönetimi veya hata kurtarma yüzünden zaman ve token tüketebilir. Daha küçük bir model ise iyi bir planlama, açık araç tanımları, test odaklı geri besleme ve kısa/doğru

context ile beklenenden iyi sonuç verebilir. Bu yüzden çalışma hem LLM kullanımını hem de agent mühendisliğini ölçer.

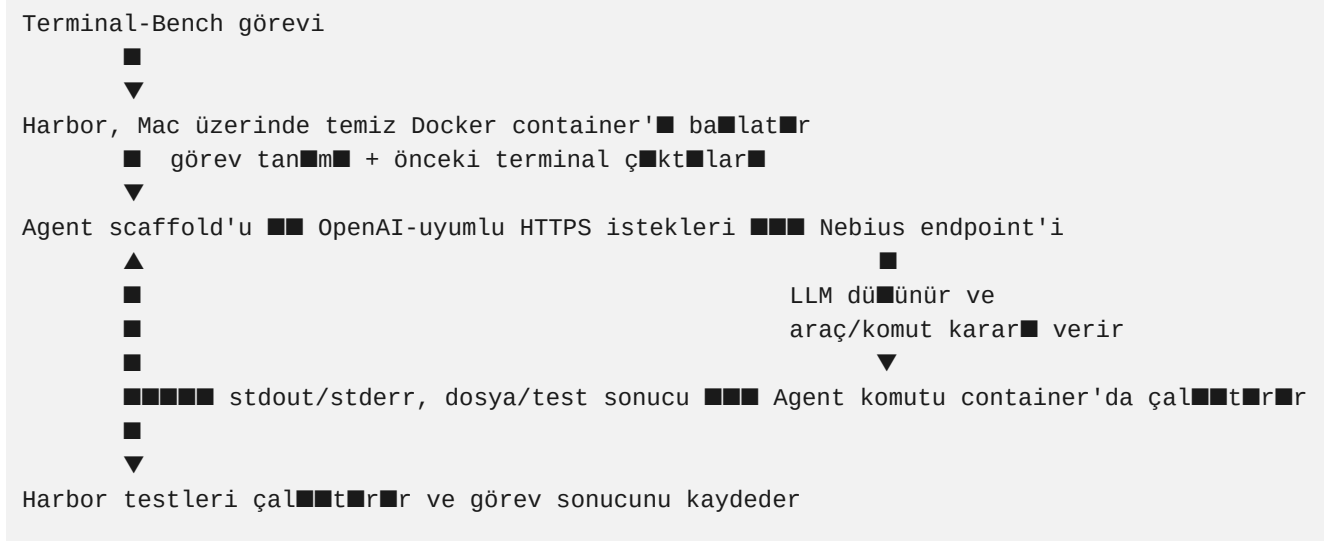
3. Mimari Genel Bakış

Sistemde iki fiziksel/mantıksal compute alanı vardır; bunları ayırmak maliyet ve kurulum kararlarını netleştirir.

Bileşen	Nerede çalışır?	Ana sorumluluk	GPU gereksinimi
Harbor + agent uygulaması	Kullanıcının Mac'i	Docker görevini açmak, komut çalıştırmak, çıktıyı modele geri vermek	Hayır
Model-serving endpoint'i	Nebius Token Factory altyapısı	Prompt'u işlemek ve model tokenlarını üretmek	Nebius tarafında

Mac'te Docker'ın kurulu olması temel ön koşulu büyük ölçüde karşılar. Harbor, görev başlatmak için Docker daemon'a erişir; her görevde taze bir container kullanıldığından CPU, disk alanı (yaklaşık 30 GB veya daha fazlası için emniyet payı) ve stabil internet gerekir. Model ağırlıkları Mac'e indirilmez, Mac üzerinde inference yapılmaz. Dolayısıyla yerel NVIDIA GPU, Apple Silicon GPU belleği ya da ayrı yerel VRAM bütçesi bu mimarinin çalışma şartı değildir.

Akış şu şekildedir:



Bu akışın kritik sonucu zaman hesabıdır: Bir turda modelin token üretmesi Nebius tarafında olur; model pytest, make, git, curl gibi bir komut önerdikten sonra bu komutun gerçek çalışma süresi Mac üzerindeki Docker host'unda geçer. Arada HTTP gecikmesi de vardır. Bunlar çoğunlukla **ardışık** maliyetlerdir; model token üretirken aynı turdaki komut henüz çalışmaz. Bu yüzden bir görevin süresi kabaca model üretimi + **arada** gecikmesi + yerel komut/test süresidir. 89 görevin tamamı tek oturuşta kısa sürede bitecek bir iş değil, saatlere hatta iterasyonlarla daha uzun süreye yayılacak bir

benchmark kampanyasıdır.

Kaggle Notebook ve Google Colab bu çalışmanın yerine geçmez: görev başına Docker container başlatma gereksinimi nedeniyle bu ortamların kısıtları resmi değerlendirme akışına uygun değildir. Notebook'lar analiz için yararlı olabilir; ana koşturma host'u değildir.

4. Elindeki Kaynakları Nasıl Kullanabilir

Nebius Token Factory: uzak inference katmanı

Nebius Token Factory (dev.nebius.com / tokenfactory.nebius.com) mevcut kredi ile OpenAI-uyumlu bir HTTP endpoint sağlar. Agent'ın model istemcisi bir base URL, API anahtarı ve model adıyla yapılandırılır; model cevapları tool-call ya da metin tabanlı komut protokolüne dönüştürülür. Yarışma kuralı belirli bir sağlayıcıyı değil, açık-ağırlıklı modelin OpenAI-uyumlu endpoint üzerinden kullanılabilmesini hedefler. Resmi örneklerde NVIDIA API Catalog ve Amazon Bedrock geçse de Nebius bu arayüz koşulunu sağladığı için uygun bir serving seçeneğidir.

Bu ayrımın ekonomik anlamı da nettir: ücret inference tokenları üzerinden Nebius'a gider; Docker komutunun CPU süresi ise yerel bilgisayardadır. Her istekte prompt, çıktı, tur sayısı, bekleme süresi ve HTTP hata oranı loglanmalıdır. Böylece yalnız toplam maliyet değil, hangi görev sınıflarının gereksiz uzun bağlam ya da tekrar deneme ürettiği de görülebilir.

LangChain: agent scaffold'unu olgunlaştırma katmanı

Resmi starter repodaki `starter/agent/agent.py`, yaklaşık 200 satırlık minimal bir ReAct döngüsü sunar: mesaj geçmişini LLM'e gönderir, cevaptan bash komutunu veya bitiş kararını ayıklar, komutu container'da çalıştırır ve sonucu yeniden geçmişe ekler. İlk doğrulama için bu sadelik bir avantajdır; fakat üretim kalitesine yaklaşan deney için sınırlıdır.

LangChain burada “sihirli şekilde daha iyi ajan” değil, sistemi düzenli kurmak için bir araçtır. Kullanılabilecek somut parçalar şunlardır:

- Terminal yürütme, dosya okuma ve gerektiğinde web araması için açık şemalı tool'lar; model hangi aracın ne yaptığını net görür.
- Planlama/uygulama ayrımı: kısa bir plan üretip her adımda doğrulama yapmak, körlemesine uzun bash blokları göndermeyi azaltır.
- Parser ve retry katmanı: geçersiz tool çağrısı, yarım kalmış JSON veya endpoint geçici hatasında kontrollü düzeltme sağlar.
- Context compaction: eski ve artık önemli olmayan stdout'u özetleyip güncel dosya durumu, başarısız test ve kararları korur.
- İzlenebilirlik: görev kimliği, tur sayısı, tool çağrıları, token kullanımı ve sonucun tek bir deney kaydında tutulmasını sağlar.

Önemli tasarım ilkesi, LangChain'i benchmark kurallarını aşmak için değil, genel amaçlı bir coding agent iskeleti kurmak için kullanmaktır. Ajanın göreve özel ipuçları gömülmemeli; aynı mekanizma farklı Terminal-Bench görevlerinde ve daha sonra kişisel projelerde çalışmalıdır.

Tavily: meşru, sınırları açık dokümantasyon araması

Tavily, agent'a sınırlı bir “web araştırma” tool'u olarak eklenebilir. Örneğin ajan bir paket hata mesajını görür, hatanın anahtar terimlerini çıkarır ve resmi dil/paket dokümantasyonu ya da projenin upstream dokümantasyonunda arama yapar. Bu, gerçek bir yazılım mühendisinin gerektiğinde docs okumasına karşılık gelir ve yarışma kuralları genel web erişimine izin verdiği için meşru bir kullanım alanıdır.

Ancak koruma katmanı şarttır: Terminal-Bench'in kendi referans çözümleri, task-specific testleri veya çözüm sızıntılarını aramak/çekmek kesinlikle yasak olmalıdır. Tool açıklamasına bu sınır yazılmalı; Terminal-Bench, task adı, reference solution, gold patch, hidden test gibi sorgular engellenmeli veya insan onayına düşmelidir. Ayrıca arama sonucundan gelen metin, güvenilmeyen dış veridir; modele “komutları körlemesine çalıştırma, yalnızca doğru” talimatı verilmelidir.

5. Model Seçimi ve Alternatifler

Birincil aday **gpt-oss-120b**'dir. Model, OpenAI tarafından Apache 2.0 ile açık-ağırlıklı yayımlanmıştır; yaklaşık 117B toplam parametrelili bir MoE mimarisi olup 128 expertten token başına dört expert aktiftir (yaklaşık 5,1B aktif parametre/token). Native MXFP4, yani 4-bit biçimde gelir. Yaklaşık 62 GB ağırlık boyutu, yarışmanın 96 GB reported-VRAM sınırının belirgin biçimde altındadır ve KV-cache/serving payı bırakır. Nebius'un bunu native biçimde sunduğu teyit edilirse ayrıca yeniden quantize etme süreci gerekmeyecektir.

Nebius fiyatlarının proje başlangıcında katalogdan doğrulanması gerekir; verilen yaklaşık değerler 1M girdi tokeni için 0,15 USD, 1M çıktı tokeni için 0,60 USD seviyesindedir. Özellikle output tokeni daha pahalı olduğundan, uzun “düşünme” döngülerini sınırlamak ve test çıktısını budamak maliyet açısından önemlidir.

İkinci, çok faydalı aday **Qwen3-30B-A3B-Instruct-2507**'dir. 30B toplam parametrelili ve daha küçük bir MoE seçeneği olarak scaffold geliştirme aşamasında hızlı/ucuz iterasyon için kullanılabilir. Önerilen strateji önce bu daha küçük modelle Harbor bağlantısı, tool protokolü, loglama ve context özetleme gibi altyapıyı oturtmak; ardından aynı deney setini gpt-oss-120b ile yeniden koşturmak. Bu, başarısızlığın “ajan mimarisi mi, model kapasitesi mi?” olduğunu ayırmayı da kolaylaştırır.

Seçenek	Güçlü yönü	Bedeli / riski	Uygun kullanım
gpt-oss-120b (MXFP4)	Karmaşık hata ayıklama ve uzun planlarda daha yüksek kapasite	Daha yavaş ve daha pahalı denemeler	Ciddi alt küme ve final benchmark
Qwen3-30B-A3B-Instruct-2507	Hızlı, düşük maliyetli scaffold iterasyonu	Karmaşık çok turlu işlerde yetersiz kalabilir	Erken geliştirme, regresyon kontrolleri
Nebius'taki diğer açık Qwen/MoE adayları	Farklı fiyat/kalite noktası	Kural uyumu ve quantization ayrıca belgelenmeli	Kontrollü A/B deneyi

Başka aday denenebilir, fakat her biri için üç şey kayıt altına alınmalıdır: açık-ağırlıklı lisans/köken, kullanılan quantization'ın en az 4-bit olması ve yayınlanan ağırlık boyutuna göre toplam reported-VRAM hesabı. Birden fazla model aynı sistemde kullanılıyorsa toplam bütçe 96 GB sınırını aşmamalıdır. Kapalı bir modelin planlama, değerlendirme veya “yalnız zor görevlerde yardımcı olma” rolü bile yasaktır; bu yüzden geliştirme sırasında dahi final ajan yoluna GPT, Claude veya Gemini çağrısı sızmamalıdır.

6. Adım Adım Yol Haritası

Bu plan, önce gözlemlenebilir doğru sistemi kurup sonra kaliteyi artırmayı hedefler. Süreler tek kişinin odaklı çalışması için kaba tahmindir; model çağrısı kuyrukları ve görev çalışma süreleri bunları uzatabilir.

- 1. Kuralları ve ortamı sabitle (yarım gün).** Kaggle sayfası, `RESOURCES.md`, submission şartları, deadline ve model sınırlarını kaydet. Docker sürümü, disk boşluğu, CPU/RAM, internet ve Nebius erişimini doğrula. API anahtarlarını `.env` içinde tut; Git'e koyma.
- 2. Harbor + starter agent smoke test'i (1 gün).** Harbor'ı ve resmi `MLM26_EfficientCoder` starter repo'sunu kur. Agent kodunu değiştirmeden, tek kolay görevde çalıştır. Hedef skor değil; container'ın açıldığını, komutun içerde çalıştığını, stdout/stderr'in geri geldiğini ve evaluator'ın sonuç ürettiğini kanıtlamaktır.
- 3. Nebius endpoint entegrasyonu (yarım–1 gün).** OpenAI-uyumlu istemciyi Nebius base URL, anahtar ve önce küçük modelle yapılandır. En az birkaç basit görevde ham istek/cevap, timeout, rate limit ve token kayıtlarını kontrol et. Bu aşamada gpt-oss-120b ile kısa bir karşılaştırma çağrısı da yapılabilir.
- 4. Ölçüm ve güvenlik tabanını ekle (1–2 gün).** Her run için model adı, quantization, commit SHA, görev kimliği, prompt/output tokenı, tur sayısı, komut süresi, test sonucu ve hata nedenini JSONL/CSV'ye yaz. Tool timeout, maksimum tur, maksimum output boyutu ve sır engelleme kurallarını belirle.
- 5. LangChain tabanlı scaffold'a geç (3–5 gün).** Minimal loop'u bir anda karmaşıklaştırmadan terminal tool'u, net system prompt, yapılandırılmış çıktı parser'ı ve kontrollü hata kurtarmayı ekle. Sonra plan/doğrulama adımları ile context compaction'ı ekle. Her değişiklikten sonra aynı küçük regresyon görev setinde önceki sürümle kıyasla.
- 6. Tavily'yi kısıtlı docs tool'u olarak ekle (1 gün).** Domain/amaç filtresi, referans çözüm/test sorgu engeli, sonuç uzunluğu sınırı ve çağrı logu koy. Web aramasını her turda varsayılan eylem değil, belirsiz API/paket davranışı için seçici araç yap.
- 7. Temsili görev alt kümesinde iterasyon (1–2 hafta).** Git, build, test, servis kurulumu ve dosya işlemleri gibi farklı sınıflardan küçük, sabit bir geliştirme seti seç. Başarı, maliyet, tur sayısı ve timeout'u birlikte incele. Başarısız görevleri göreve özel kural yazarak değil, genel prompt/tool/context tasarımını iyileştirerek ele al.
- 8. Tüm 89 görev için kontrollü kampanya (birkaç gün–2 hafta).** Önce küçük modelle geniş sağlık kontrolü, sonra seçilen final konfigürasyonla tam koşu yap. Süreç kesintiye dayanıklı olmalı:

tamamlanan görevler kayıtlı kalmalı, yalnızca başarısız/yarım görevler yeniden çalıştırılabilir. Görev başı 60 dakika üst sınırına uygun timeout ve retry politikası kullan.

9. **Maliyet, token ve skor analizi (1–2 gün).** TB score, görev sınıfına göre başarı, token cezası, dolar maliyeti ve en yavaş komutları raporla. Aynı scaffold için Qwen ve gpt-oss sonuçlarını karşılaştır; pahalı modelin getirdiği başarı artışının maliyete değip değmediğine karar ver.
10. **Açık kaynak teslim paketi (1–2 gün).** Temiz bir public GitHub repo, sabit commit/tag, kurulum talimatı, çevresel değişken örneği, mimari notu, model/quantization beyanı ve reproducible run komutları hazırla. Kaggle Writeup'a yöntem, ölçülen skor, maliyet yaklaşımı, sınırlamalar ve no-leakage yaklaşımını yaz; submission öncesi kuralları tekrar doğrula.

7. Öngörülen Zorluklar ve Verimlilik (Efficiency) Riskleri

En görünür risk süre tahminidir. 89 görevin her biri basit değildir ve görevin kendi test/derleme süresi uzun olabilir. Buna modelin token üretimi, ağ turu ve agent'ın deneme-yanılma turları eklenir. Tek bir görevde 10–20 tur yaşanır, her turdaki küçük gecikmeler anlamlı toplam süreye dönüşür. Görev başına 60 dakikalık limit de ajanı “biraz daha denersem çözerim” davranışından koruyan gerçek bir sınırdır.

Uzun agent koşullarında context window şişmesi ikinci ana risktir. Büyük test logları, find çıktıları veya tekrar eden hata mesajları geçmişe ham biçimde eklenirse model kritik bilgiyi kaybedebilir, önceki yanlış varsayımını pekiştirip döngüye girebilir. Çözüm, output truncation, görev durum özeti, son hatanın korunması, başarısız hipotez listesi ve maksimum tur gibi mekanizmalardır. Özet çıkarma da bilgiyi kaybettirebileceği için “çalışan komutlar / değişen dosyalar / son test sonucu / açık hipotez” gibi yapılandırılmış bir özet tercih edilmelidir.

Daha küçük model maliyeti düşürür ama karmaşık, çok araçlı görevlerde yanlış teşhis ve erken teslim riski büyür. Daha büyük model ise doğru çözüme daha sık yaklaşırsa bile yavaş/pahalı olabilir ve kötü scaffold'u otomatik düzeltmez. Bu nedenle model kararını hissiyatla değil, temsili bir görev setindeki başarı-maliyet-tur sayısı üçlüsüyle vermek gerekir.

Docker tarafında disk imajlarının birikmesi, container temizliği, RAM baskısı, CPU yoğun testlerin Mac'i yavaşlatması ve ağ kopmaları operasyonel risklerdir. macOS üzerindeki Docker Linux VM katmanı nedeniyle bazı Linux-spesifik görevlerde native Linux'a göre performans ya da davranış farkı görülebilir. İlk smoke test'te ve çeşitli görev türlerinde bunu ölçmek; disk kullanımını izlemek; başarısız container/koşu loglarını saklamak önemlidir. Gerekirse yalnız benchmark host'u olarak bir Linux makine düşünülür, fakat başlangıç için ayrı GPU host'u gerekli değildir.

Son olarak API maliyeti, özellikle output tokenları, beklentiyi aşabilir. Yarışma skorunda token israfı için küçük ve tavanlı bir ceza vardır: görev başına 100M token başına 0,01 puan. Bu, birincil hedefin doğru çözüm olduğunu; token optimizasyonunun ise daha çok maliyet kontrolü ve yakın skorlarda tie-breaker olduğunu gösterir. Ucuzluk uğruna test yapmayı kesmek yanlış optimizasyondur; gereksiz tekrar ve devasa logları azaltmak doğru optimizasyondur.

8. Bu Proje Sana Öğrenimin Açısından Ne Katar (yarışma sonucundan bağımsız)

Bu proje, LangChain'i bir sohbet botuna bağlamaktan çok daha somut biçimde öğretir: model kararını araç şemasına dönüştürme, tool sonucunu yönetme, geçersiz çıktıyı toplama, state taşımak, context budamak ve gözlemlenebilir deney tasarlamak gerekir. Örneğin bir `pytest` sonucunu salt metin olarak tekrar modele vermek yerine, “hangi test başarısız, hangi dosya değişti, sıradaki doğrulama ne?” durumuna dönüştürmeyi öğrenirsin. Bu, kurumsal agent iş akışlarında doğrudan kullanılan bir pratiktir.

Açık-ağırlıklı model serving tarafında, “parametre sayısı” ile gerçek serving maliyeti, quantization, MoE aktif parametre sayısı, KV-cache ve endpoint gecikmesi arasındaki ilişkiyi uygulamalı görürsün. Bir modelin teknik olarak güçlü olmasının, ajan görevi için otomatik olarak ekonomik veya hızlı olacağı anlamına gelmediğini ölçerek öğrenirsin. Nebius üzerinden bu deney, yerel GPU kümesi kurmadan bile gerçekçi bir inference mimarisi kurmayı sağlar.

Harbor ve Terminal-Bench ise evaluator bakış açısı kazandırır. Başarılı demo ile tekrar üretilebilir başarı arasındaki farkı; temiz ortam, gizli test, timeout, loglama ve benchmark leakage'ın neden önemli olduğunu içeriden görürsün. Bu deneyim, kestirimci bakım veya veri bilimi projelerinde de model değerlendirme boru hatları, deney izleme ve güvenilir otomasyon tasarlamaya taşınabilir.

Docker tabanlı izole test ortamları kurmak da ayrı bir kazanımdır. Bir agent'ın host sisteme değil görev container'ına etki etmesi, bağımlılıkların tekrar üretilebilirliği, dosya sistemi sınırları ve komut timeout'ları pratikte çok değerlidir. Sonuçta bu çalışma, mevcut dual-agent orkestrasyonu deneyimini kapalı model araçlarından açık model, ölçüm odaklı ve denetlenebilir bir sistem tasarımına genişletir.

9. Sonuçta Ortaya Çıkacak Ürün

Proje bittiğinde somut çıktı birden fazladır. İlk olarak, Nebius üzerinden açık-ağırlıklı bir modeli kullanan, Harbor/Terminal-Bench container'larında terminal araçlarını çalıştıran, LangChain tabanlı ve açık kaynaklı bir coding agent deposu ortaya çıkar. Repo; kurulum, yapılandırma, güvenlik sınırları, model quantization beyanı, deney komutları ve loglama yaklaşımıyla başkası tarafından da çalıştırılabilir olmalıdır.

İkinci olarak, hangi commit ve model konfigürasyonu ile alındığı belli, ölçülmüş bir Terminal-Bench skoru elde edilir. Skora maliyet, token, tur sayısı ve görev sınıfı analizi eşlik eder; bu yüzden sayı tek başına değil, neyin işe yarayıp yaramadığını anlatan bir mühendislik sonucu olur. Üçüncü çıktı Kaggle Writeup ve public GitHub submission'ıdır. En değerli kalıcı çıktı ise yarışmadan bağımsızdır: kişisel projelerde uyarlanabilecek bir **LangChain + Nebius agent scaffold** şablonu.

10. Teknik ve Kısıtsal Detaylar (Özet Tablo)

Konu	Kesin/pratik kural
Model türü	Yalnız açık-ağırlıklı LLM kullanılabilir.

Konu	Kesin/pratik kural
Kapalı model yasağı	GPT, Claude, Gemini vb. kapalı modeller planlayıcı dahil hiçbir rolde kullanılamaz.
VRAM bütçesi	Kullanılan ağırlıkların yayınlanmış boyutuna göre toplam reported VRAM en fazla 96 GB olmalıdır.
Quantization	Model ağırlıkları en az 4-bit quantize olmalıdır.
Değerlendirme	Terminal-Bench 2.1'de 89 Docker tabanlı görev; Harbor resmi çalışma/evaluasyon katmanıdır.
Süre sınırı	Bir görev için en fazla yaklaşık 60 dakika; güncel limit yarışma belgelerinden teyit edilmelidir.
Skor	$\text{leaderboard_score} = \text{TB_score} - \text{token_israf} \times \text{cezas}$; TB score önceliklidir, token cezası küçüktür ve tavanlıdır.
Hardcoding yasağı	Göreve özel, benchmark cevabını hedefleyen kod/ipuçları yasaktır; kod genel amaçlı olmalıdır.
Eğitim/sızıntı yasağı	Skorlanan görevler üzerinde eğitim yapmak, referans çözüm veya testleri web'den edinmek yasaktır.
Web erişimi	Genel dokümantasyon ve paket bilgisi için serbesttir; Terminal-Bench çözümü/testi aramak yasaktır.
Submission	Dosya yüklemek yerine Kaggle Writeup ve belirli commit/tag'e işaret eden public GitHub repo sunulur.
İnceleme	En iyi 5 submission, deadline sonrasında yeniden koşturma ve kod incelemesinden geçer.
Ödül	Nakit ödül yoktur; Kudos ve ML+X showcase daveti olasılığı vardır.
Deadline	Yaklaşık üç ay/rolling görünümü verilmiştir; kesin tarih yarışma sayfasından submission öncesi teyit edilmelidir.

Kuralları tek cümlede özetlersek: **96 GB altında, en az 4-bit açık-ağırlıklı model(ler)le, kapalı model yardımı veya görev/test çözümü sızıntısı olmadan, genel amaçlı ve açık kaynak bir agent üret; bunu kamuya açık bir repo ve Kaggle Writeup ile tekrar üretilbilir biçimde teslim et.**

11. Kapanış / Tavsiye

Evet, bu proje buna deęer; özellikle hedef yarışma kupası deęil, geręek bir coding agent'ı ölçerek inşa etmek ve LangChain/Nebius/Harbor ekseninde derinleşmekse. Rekabetin řu an düşük görünmesi ve nakit ödölün olmaması, projeyi daha da uygun bir öğrenme laboratuvarına çeviriyor: aceleyle leaderboard kovalamak yerine sağlam altyapı, deney disiplini ve tekrar kullanılabilir scaffold öncelik olabilir.

Başlama sırası net olmalı: önce Docker + Harbor + deęiştirilmemiş starter agent ile tek görevlik smoke test; sonra Nebius'u küçük Qwen modeliyle bağlama; ölçüm/loglama temelini kurma; LangChain scaffold'u ve kısıtlı Tavily docs tool'unu ekleme; temsilî görevlerde iterasyon; en son gpt-oss-120b ile tam deęerlendirme. Bu sıra, en pahalı denemeleri sistem henüz kırılganken yapmak yerine, önce her katmanın çalıştığını kanıtlar. Başarı 89/89 olmasa bile ortaya çıkacak repo, benchmark sonuçları ve reusable agent şablonu başlı başına güçlü ve geręek dünyaya taşınabilir bir çıktıdır.